CSCE883 Machine Learning **Name:** *Taylor Belcher*
Homework 3
10/11/2026

---

1. *(Question 1.)* Gradient propagation and CNN dimensions.

   A. *Gradient propagation.* We have a linear network with L layers with the initial input $h_0 = x$ and at each subsesquent layer l (where $1 \leq l \leq L$), $h_l = \sigma(w_l h_{l-1})$ and $\sigma(z) = \dfrac{1}{1+e^{-z}}$. $|w_l| \leq 1$ and there are no biases.

   (i) We want to estimate an upper bound on $\left|\dfrac{\partial h_L}{\partial x}\right|$ in terms of L. We have that

   $$\begin{aligned} h_L &= \sigma(w_L h_{L-1}) \\ &= \sigma\big(w_L \sigma\big(w_{L-1}\sigma(\cdots\sigma(w_1 x))\big)\big). \end{aligned}$$

   At any given level $l \geq 2$, let $g_l(x)$ denote the remaining nested expression, including the weight w_l, all the way down to the input x:

   $$g_l(x) = w_l \sigma\big(w_{l-1}\sigma(\cdots\sigma(w_1 x))\big), \qquad g_1(x) = w_1 x.$$

   Thus $h_l = \sigma(g_l(x))$. By the chain rule, the partial derivative of h_L with respect to x can be written as

   $$\begin{aligned} \left|\frac{\partial h_L}{\partial x}\right| &= |\sigma'(g_L(x))\, g_L'(x)| \\ &= |\sigma'(g_L(x))|\ \left|w_L\, \sigma'(g_{L-1}(x))\, g_{L-1}'(x)\right| \\ &= |\sigma'(g_L(x))|\ |w_L\, \sigma'(g_{L-1}(x))| \\ &\quad \cdot |w_{L-1}\, \sigma'(g_{L-2}(x))| \cdots |w_2\, \sigma'(g_1(x))|\ |w_1| \\ &= \left(\prod_{l=1}^{L} |w_l|\right)\left(\prod_{l=1}^{L} |\sigma'(g_l(x))|\right) \\ &\leq 1 \cdot \prod_{l=1}^{L} |\sigma'(g_l(x))|\,. \end{aligned}$$

   The inequality follows from $|w_l| \leq 1$ for every layer, so $\prod_{l=1}^{L} |w_l| \leq 1$. Now the derivative of the sigmoid activation function is

   $$\sigma'(z) = -(1+e^{-z})^{-2}(-e^{-z}) = \frac{e^{-z}}{(1+e^{-z})^2}.$$

   To bound this derivative, observe that

   $$1 - \sigma(z) = 1 - \frac{1}{1+e^{-z}} = \frac{e^{-z}}{1+e^{-z}}.$$

Thus we can rewrite the derivative as

$$\sigma'(z) = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(z)\big(1-\sigma(z)\big).$$

Since $0 < \sigma(z) < 1$, this derivative is positive. Completing the square gives

$$|\sigma'(z)| = \sigma(z)\big(1-\sigma(z)\big) = \frac{1}{4} - \left(\sigma(z) - \frac{1}{2}\right)^2 \leq \frac{1}{4}.$$

Applying this bound to each of the L factors in our product yields

$$\begin{aligned}\left|\frac{\partial h_L}{\partial x}\right| &\leq \prod_{l=1}^{L} |\sigma'(g_l(x))| \\ &\leq \prod_{l=1}^{L} \frac{1}{4} \\ &= \boxed{\left(\frac{1}{4}\right)^L = 4^{-L}}.\end{aligned}$$

Therefore the magnitude of the input derivative is bounded above by 4^{-L}, which decreases exponentially with the number of layers. Increasing the number of layers will decrease the gradient exponentially, making it so that earlier layers will take much longer to train.

(ii) If the weights were not bounded above by 1, then we could counter-balance the derivative of the sigmoid function with a larger product of weights, but this is not a guarantee as the weights would also increase the input to the sigmoid functions and the larger the input, the flatter the curve on the sigmoid gets, which means the derivative would also be getting smaller as it moves into the saturation region. So increasing the weights can help with a vanishing gradient, but it is not a guaranteed solution. More precisely, the weight bound matters because each layer contributes both a weight and a sigmoid derivative:

$$\left|\frac{\partial h_l}{\partial h_{l-1}}\right| = |w_l \sigma'(w_l h_{l-1})| \leq \frac{|w_l|}{4}.$$

With $|w_l| \leq 1$, each factor is at most $1/4$, giving the exponential bound in part (i). Without this weight bound, larger weights could offset small sigmoid derivatives, so small sigmoid derivatives alone do not prove that every deep network must have vanishing gradients. However, larger weights can also move the sigmoid inputs into regions where its derivative is small. Increasing the weights is therefore not a guaranteed solution.

A vector residual block adds its input back to a learned transformation of that input:

$$\mathbf{h}_{l+1} = \mathbf{h}_l + F_l(\mathbf{h}_l), \qquad F_l : \mathbb{R}^d \to \mathbb{R}^d.$$

Its Jacobian contains the partial derivative of each output component with respect to each input component. Writing the ith output component as $h_{l+1,i} = h_{l,i} + F_{l,i}(\mathbf{h}_l)$ gives

$$\begin{aligned}[J_l]_{ij} &= \frac{\partial h_{l+1,i}}{\partial h_{l,j}} \\ &= \frac{\partial h_{l,i}}{\partial h_{l,j}} + \frac{\partial F_{l,i}(\mathbf{h}_l)}{\partial h_{l,j}} \\ &= \delta_{ij} + [J_{F_l}(\mathbf{h}_l)]_{ij},\end{aligned}$$

where $\delta_{ij} = 1$ if $i = j$ and 0 otherwise, is Kroenecker's delta. These entries form the identity matrix, so

$$\boxed{J_l = I + J_{F_l}(\mathbf{h}_l)}.$$

For a scalar loss $\mathcal{L}$, the chain rule propagates its gradient backward through the block as

$$\nabla_{\mathbf{h}_l}\mathcal{L} = \big(I + J_{F_l}(\mathbf{h}_l)\big)^{\mathsf{T}}\nabla_{\mathbf{h}_{l+1}}\mathcal{L}.$$

The identity term provides a direct contribution from the next layer's gradient. When J_{F_l} is small, the block's Jacobian is close to I, which helps preserve gradients. This does not guarantee successful training: for example, $J_{F_l} = -I$ would cancel the identity term entirely, and products of Jacobians across many blocks can still shrink or amplify gradients.

B. *CNN dimensions and parameter counts.*

The input is $3 \times 24 \times 24$ in channel–height–width order. Convolutions have stride 1, no padding, dilation 1, and one bias per output channel. Pooling uses a 2×2 window with stride 2, no padding, and floor rounding.

(i) For either spatial dimension n, the output-size formula is

$$n_{\text{out}} = \left\lfloor \frac{n_{\text{in}} + 2p - d(k-1) - 1}{s} \right\rfloor + 1,$$

where k is the kernel size, p is padding, d is dilation, and s is stride. Here $p = 0$ and $d = 1$, so this becomes $n_{\text{out}} = \lfloor (n_{\text{in}} - k)/s \rfloor + 1$. Since height and width are equal, the same calculation applies to both:

Layer	Spatial calculation	Output dimension
Conv 1	$\lfloor (24-3)/1 \rfloor + 1 = 22$	$12 \times 22 \times 22$
Pool 1	$\lfloor (22-2)/2 \rfloor + 1 = 11$	$12 \times 11 \times 11$
Conv 2	$\lfloor (11-3)/1 \rfloor + 1 = 9$	$128 \times 9 \times 9$
Pool 2	$\lfloor (9-2)/2 \rfloor + 1 = 4$	$128 \times 4 \times 4$

A convolution's channel count is its number of output filters, while pooling preserves the channel count. In Pool 2, floor rounding gives 4, so the last row and column of the 9×9 input are not covered by a pooling window.

(ii) Since groups $= 1$, each convolutional output filter spans all input channels. Therefore a $k \times k$ filter has $C_{\text{in}}k^2$ weights and one bias, giving

$$N_{\text{conv}} = C_{\text{out}} \cdot (C_{\text{in}}k^2 + 1).$$

For Conv 1, each output filter has $3 \cdot 3 \cdot 3 = 27$ weights and one bias. With 12 output filters, the total is

$$N_{\text{Conv 1}} = 12(27 + 1) = \boxed{336} \quad \text{(324 weights and 12 biases).}$$

Conv 2 receives 12 channels, so each output filter has $12 \cdot 3 \cdot 3 = 108$ weights and one bias. With 128 output filters,

$$N_{\text{Conv 2}} = 128(108 + 1) = \boxed{13{,}952} \quad \text{(13,824 weights and 128 biases).}$$

Max pooling only selects the maximum within each window and has no trainable weights or biases. Thus Pool 1 and Pool 2 each have $\boxed{0}$ trainable parameters.

(iii) After Pool 2, flattening the $128 \times 4 \times 4$ tensor gives

$$n_{\text{flat}} = 128 \cdot 4 \cdot 4 = \boxed{2{,}048}$$

features. A fully connected layer with 10 outputs has one weight for each input–output pair and one bias per output, so it adds

$$N_{\text{FC}} = 2{,}048 \cdot 10 + 10 = \boxed{20{,}490}$$

trainable parameters. Flattening itself has no trainable parameters.

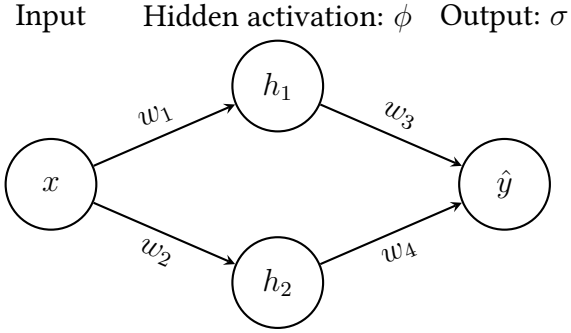
In a convolution, the same filter weights and bias are reused at every spatial location. Thus its parameter count depends on the kernel size and channel counts, rather than the number of output locations. A fully connected mapping learns separate weights for every input–output pair. For example, a fully connected mapping from the original $3 \cdot 24 \cdot 24 = 1{,}728$ inputs to the $12 \cdot 22 \cdot 22 = 5{,}808$ outputs of Conv 1 would require

$$1{,}728 \cdot 5{,}808 + 5{,}808 = 10{,}042{,}032$$

parameters, compared with Conv 1's 336. This reduction comes from both local connectivity (each output uses a 3×3 spatial patch) and weight sharing (all locations use the same filters).

2. *(Question 2.)* We are given the following network and constraints.

   For one example (x, y), where $y \in \{0, 1\}$, all four weights are scalars and there are no biases. The hidden activation ϕ changes between parts, while the output activation is always sigmoid.



The forward pass is

$$\begin{aligned} h_1 &= \phi(w_1 x), & h_2 &= \phi(w_2 x), \\ z &= w_3 h_1 + w_4 h_2, & \hat{y} &= \sigma(z), \end{aligned}$$

with binary cross-entropy loss

$$\mathcal{L} = -y \log(\hat{y}) - (1-y)\log(1-\hat{y}).$$

We use the natural logarithm for calculations.

A. Let $\phi = \sigma$. Then since $\hat{y} = \sigma(z)$, the partial derivative of the loss function $\mathcal{L}$ with respect to z is:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \hat{y}} &= -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}, \\ \frac{\partial \hat{y}}{\partial z} &= \hat{y}(1-\hat{y}), \\ \frac{\partial \mathcal{L}}{\partial z} &= \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \\ &= \left(-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} \right) \hat{y}(1-\hat{y}) \\ &= -y(1-\hat{y}) + (1-y)\hat{y} = \boxed{\hat{y} - y}. \end{aligned}$$

We have that $z = w_3 h_1 + w_4 h_2$. Therefore:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_3} &= \frac{\partial \mathcal{L}}{\partial z} \frac{\partial z}{\partial w_3} = (\hat{y} - y) h_1 \\ &= \boxed{\big[\sigma(w_3 \sigma(w_1 x) + w_4 \sigma(w_2 x)) - y\big] \sigma(w_1 x)}, \\ \frac{\partial \mathcal{L}}{\partial w_4} &= \frac{\partial \mathcal{L}}{\partial z} \frac{\partial z}{\partial w_4} = (\hat{y} - y) h_2 \\ &= \boxed{\big[\sigma(w_3 \sigma(w_1 x) + w_4 \sigma(w_2 x)) - y\big] \sigma(w_2 x)}. \end{aligned}$$

Let $a_1 = w_1 x$ and $a_2 = w_2 x$, with $h_1 = \sigma(a_1)$ and $h_2 = \sigma(a_2)$.

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_1} &= \frac{\partial \mathcal{L}}{\partial z} \frac{\partial z}{\partial h_1} \frac{\partial h_1}{\partial a_1} \frac{\partial a_1}{\partial w_1} \\ &= (\hat{y} - y) \cdot w_3 \cdot h_1(1-h_1) \cdot x = (\hat{y} - y) w_3 h_1 (1-h_1) x. \end{aligned}$$

Therefore,

$$\boxed{\frac{\partial \mathcal{L}}{\partial w_1} = \big[\sigma(w_3 \sigma(w_1 x) + w_4 \sigma(w_2 x)) - y\big] w_3 \sigma(w_1 x) \big(1 - \sigma(w_1 x)\big) x}.$$

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_2} &= \frac{\partial \mathcal{L}}{\partial z} \frac{\partial z}{\partial h_2} \frac{\partial h_2}{\partial a_2} \frac{\partial a_2}{\partial w_2} \\ &= (\hat{y} - y) \cdot w_4 \cdot h_2(1-h_2) \cdot x = (\hat{y} - y) w_4 h_2 (1-h_2) x. \end{aligned}$$

Therefore,

$$\boxed{\frac{\partial \mathcal{L}}{\partial w_2} = \big[\sigma(w_3\sigma(w_1x) + w_4\sigma(w_2x)) - y\big] w_4\sigma(w_2x)\big(1 - \sigma(w_2x)\big)x}.$$

With learning rate $\eta > 0$, the simultaneous gradient-descent updates are

$$\begin{aligned} w_1^{\text{new}} &= w_1 - \eta(\hat{y} - y)w_3h_1(1 - h_1)x, \\ w_2^{\text{new}} &= w_2 - \eta(\hat{y} - y)w_4h_2(1 - h_2)x, \\ w_3^{\text{new}} &= w_3 - \eta(\hat{y} - y)h_1, \\ w_4^{\text{new}} &= w_4 - \eta(\hat{y} - y)h_2. \end{aligned}$$

B. Let

$$\phi(u) = \frac{\log(1 + e^{2ku})}{2k}, \qquad k > 0.$$

Differentiating gives

$$\begin{aligned} \phi'(u) &= \frac{1}{2k} \cdot \frac{1}{1 + e^{2ku}} \cdot e^{2ku} \cdot 2k \\ &= \frac{e^{2ku}}{1 + e^{2ku}} = \frac{1}{1 + e^{-2ku}} = \boxed{\sigma(2ku)}. \end{aligned}$$

Thus the derivative is sigmoid evaluated at $2ku$, while the hidden outputs remain $h_1 = \phi(w_1x)$ and $h_2 = \phi(w_2x)$. The output activation and loss are unchanged, so $\partial\mathcal{L}/\partial z = \hat{y} - y$. The weight gradients are

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_1} &= (\hat{y} - y)w_3\phi'(w_1x)x = \boxed{(\hat{y} - y)w_3\sigma(2kw_1x)x}, \\ \frac{\partial \mathcal{L}}{\partial w_2} &= (\hat{y} - y)w_4\phi'(w_2x)x = \boxed{(\hat{y} - y)w_4\sigma(2kw_2x)x}, \\ \frac{\partial \mathcal{L}}{\partial w_3} &= (\hat{y} - y)h_1 = \boxed{(\hat{y} - y)\phi(w_1x)}, \\ \frac{\partial \mathcal{L}}{\partial w_4} &= (\hat{y} - y)h_2 = \boxed{(\hat{y} - y)\phi(w_2x)}, \end{aligned}$$

where $\hat{y} = \sigma\big(w_3\phi(w_1x) + w_4\phi(w_2x)\big)$. Our chosen $\phi(u)$ approaches ReLU as k goes to infinity. For fixed $u > 0$, the limit has the indeterminate form ∞/∞. Applying L'Hôpital's rule with respect to k gives

$$\begin{aligned} \lim_{k\to\infty} \phi(u) &= \lim_{k\to\infty} \frac{\log(1 + e^{2ku})}{2k} \\ &= \lim_{k\to\infty} \frac{2ue^{2ku}/(1 + e^{2ku})}{2} \\ &= \lim_{k\to\infty} \frac{u}{1 + e^{-2ku}} = u. \end{aligned}$$

For fixed $u < 0$, $e^{2ku} \to 0$, so

$$\lim_{k\to\infty} \phi(u) = \lim_{k\to\infty} \frac{\log(1 + e^{2ku})}{2k} = 0.$$

At $u = 0$,

$$\lim_{k\to\infty} \phi(0) = \lim_{k\to\infty} \frac{\log 2}{2k} = 0.$$

Therefore,

$$\boxed{\lim_{k\to\infty} \phi(u) = \begin{cases} u, & u > 0, \\ 0, & u \leq 0 \end{cases} = \max(0, u) = \mathrm{ReLU}(u)}.$$

ReLU is differentiable for $u \neq 0$, with derivative 0 for $u < 0$ and 1 for $u > 0$. At $u = 0$, its ordinary derivative does not exist, but we can choose a value for the purposes of backpropogation. Since $u = 0$ is the boundary between a zero and non-zero derivative, a choice of 0 for the derivative makes sense, but we could choose another value between 0 and 1 instead. This supplies a value for the chain-rule factor when a unit's input is exactly zero, allowing backpropagation to proceed. Choosing 1 is also valid, but does not change the zero derivative for negative inputs. Thus the smooth activation ϕ is an alternative to ReLU, rather than a prerequisite for training with ReLU.

C. We set

$$x = 0.7, \qquad y = 1, \qquad (w_1, w_2, w_3, w_4) = (0.2, -0.3, 0.4, -0.5).$$

For sigmoid and scaled Softplus with $k = 1$, and compare the analytic gradients with automatic differentiation and central finite differences using $\varepsilon = 10^{-5}$ and float64 arithmetic. The four gradients and the maximum absolute discrepancy for each comparison are reported below. The code is in `hw3_question2c-tbelcher.ipynb` and uses NumPy 1.26.4 and Autograd 1.9.1. The loss is evaluated directly from logits as $\log(1 + e^z) - yz$ using `logaddexp` for numerical stability.

Sigmoid hidden activation

Weight	Analytic	Autodiff	Central difference
w_1	-0.035000918963	-0.035000918963	-0.035000918963
w_2	0.043484696482	0.043484696482	0.043484696477
w_3	-0.268791282082	-0.268791282082	-0.268791282082
w_4	-0.224950589689	-0.224950589689	-0.224950589683

Scaled Softplus hidden activation ($k = 1$)

Weight	Analytic	Autodiff	Central difference
w_1	-0.078049347229	-0.078049347229	-0.078049347224
w_2	0.067922216435	0.067922216435	0.067922216440
w_3	-0.206270074531	-0.206270074531	-0.206270074532
w_4	-0.123587760488	-0.123587760488	-0.123587760492

The maximum absolute discrepancies across the four weights, computed before rounding the table entries, are

Activation	Analytic vs. autodiff	Analytic vs. central difference
Sigmoid	5.551115×10^{-17}	5.799611×10^{-12}
Scaled Softplus	0	5.580245×10^{-12}

Both comparisons agree closely with the analytic gradients. For each weight w_i, central finite differences compute the average rate of change between $w_i - \varepsilon$ and $w_i + \varepsilon$, holding all other weights fixed:

$$\frac{\partial \mathcal{L}}{\partial w_i} \approx \frac{\mathcal{L}(w_i + \varepsilon) - \mathcal{L}(w_i - \varepsilon)}{2\varepsilon}.$$

Here $\mathcal{L}(w_i \pm \varepsilon)$ denotes the loss after changing only w_i. A smaller ε makes this average rate of change closer to the derivative in exact arithmetic. However, float64 stores each computed loss with finite precision. As ε decreases, the two loss values become nearly equal, and their difference can become comparable to their rounding errors. Dividing that difference by the small number 2ε amplifies those errors. Thus reducing ε can initially improve agreement, but an excessively small step can worsen it.

3. *(Question 3.)* Evaluation protocol audit.

   (a) The preprocessing leakage occurs because the student includes the test observations when computing the means and standard deviations used for standardization. These parameters should be estimated from the training set only, then applied to the validation and test sets. Otherwise, information from the test set influences the model's preprocessing.

   Additionally, the student uses the test set to choose the best of 20 architectures. Test scores depend partly on the particular observations in the test set, so selecting the highest score can favor a model that performs unusually well on those observations by chance. The selected score is consequently optimistic as an estimate of performance on new data. Repeating the procedure five times does not remove either problem: each repetition still leaks test information and selects a model using its test performance.

   (b) We should first split the data into training, validation, and test sets, for example using an $80/10/10$ split. Fit the standardization parameters on the training set only and use those same parameters to transform all three sets. Train the candidate models on the training set and use validation performance to tune hyperparameters and select the architecture.

   For early stopping, monitor validation loss during training. Stop when it has not improved for a prespecified number of epochs, and restore the checkpoint with the lowest validation loss. This selects the training duration using validation data rather than test data. After choosing the model with the best validation performance and fixing all modeling choices, evaluate it on the held-out test set and report its performance without using that result for further tuning or selection.

(c) If several rows measure the same material, placing those rows in both training and testing can make test performance optimistic for prediction on previously unseen materials. The model may benefit from having already seen related measurements of a test material during training. We should instead split by material: all rows from a given material must belong entirely to the training, validation, or test set. This also prevents validation-based selection from benefiting from overlap with training materials. We need a material identifier for each row that reliably groups repeated measurements of the same material, including records that use different names for that material.

4. *(Question 4.)* DNA composition and sequence order.

A. *Hypothesis and data preparation.* My hypothesis is that sequence order provides more information for distinguishing the two classes than nucleotide composition. DNA uses a shared alphabet, and I expect the arrangement of those letters to be more informative than their overall frequencies. This is a prediction about the supplied classification task; sharing an alphabet does not rule out differences in composition. Before training, I predicted that frequency-only logistic regression would perform near chance (50%), that sequence models would perform better on intact sequences and lose accuracy after shuffling, and that adding neighboring-base information would improve on frequencies alone. A strong frequency baseline, little neural-model degradation after shuffling, or no improvement from neighboring-base features would challenge these predictions.

The audit found 20,000 aligned sequence–label records, each with 250 bases from A, C, G, T, and 10,000 records in each numeric class. There were no exact duplicate sequences or conflicting labels. Numeric labels were retained without assigning a biological identity to either value. With each split seed $s \in \{11, 22, 33\}$, I first held out 5,000 stratified test rows, then split the development rows into 12,000 training and 3,000 validation rows. The same indices were used for every compared model.

One-hot encoding uses channel order A, C, G, T. The MLP receives 1,000 flattened entries, and the CNN receives a 4×250 tensor. Nucleotide frequencies are counts divided by 250; the additional 16 dinucleotide frequencies are adjacent-pair counts divided by 249. These deterministic features require no learned preprocessing, and no scaling was applied. Pair order is AA, AC, AG, AT, CA, CC, CG, CT, GA, GC, GG, GT, TA, TC, TG, TT.

B. *Models and shuffling control.* Both logistic regressions use L2 regularization with $C = 1$, an intercept, the `lbfgs` solver, and at most 1,000 solver iterations; all fits converged within that limit. The frequency model has 5 parameters, and the frequency-plus-pairs model has 21. The neural architectures below include biases; ReLU, pooling, and flattening have no trainable parameters.

MLP layer	Output dimension	Parameters
Flatten	1,000	0
Linear + ReLU	64	64,064
Linear + ReLU	32	2,080
Linear (logit)	1	33
Total		66,177

CNN layer	Output dimension	Parameters
Conv 1 + ReLU	16×250	464
Max pool 1	16×125	0
Conv 2 + ReLU	32×125	3,616
Max pool 2	32×62	0
Flatten	1,984	0
Linear + ReLU	32	63,520
Linear (logit)	1	33
Total		67,633

CNN convolutions run along sequence positions, with kernel width 7, stride 1, padding 3, dilation 1, and groups 1. Pooling has window and stride 2, no padding, and floor rounding. Similar parameter totals do not make the architectures equivalent in capacity.

Before fitting, I declared one training configuration for each neural model: Adam, learning rate 0.001, batch size 128, zero weight decay, and no dropout. The loss is binary cross-entropy computed directly from logits. Models train for at most 100 epochs with patience 10 on validation loss and restore the checkpoint with the lowest validation loss separately in each split. Initialization seeds are $s + 1000$ and data-loader seeds are $s + 2000$. All models and checkpoints were fixed before inspecting any test results. Probabilities at least 0.5 predict class 1.

For the control, positions within each test sequence were independently shuffled using seed $s + 100$, with identical shuffled sequences supplied to all models. Labels and nucleotide counts were preserved, and frozen models were evaluated without retraining. The frequency baseline isolates prediction from overall nucleotide composition. Its performance above chance establishes that composition contains information useful to this classifier on these splits. It does not establish that composition is the only useful information, and poor performance would not rule out composition information that a different classifier could exploit. Its predictions must remain unchanged after shuffling because its four input features are identical; this invariance also checks that the shuffling control preserves composition.

Frozen-model shuffling tests whether a trained model's predictions and accuracy depend on the arrangement of bases while holding their counts fixed. A decrease in accuracy establishes sensitivity to the ordering patterns disrupted by this perturbation, rather than to a change in nucleotide composition. It does not separate loss of useful order information from difficulty processing inputs outside the model's training distribution, nor does it measure the best accuracy achievable by a model retrained on

shuffled data. Little or no accuracy change would not prove that the model ignores order: predictions can change without changing overall accuracy. Neither control establishes biological mechanisms or generalization to new genomic regions or sequence families.

C. *Neighboring-base investigation.* I compared two feature settings: four nucleotide frequencies and those same four features plus 16 dinucleotide frequencies. Splits, logistic-regression settings, and evaluation were held fixed. This tests whether adjacent-pair statistics add predictive information beyond overall composition. The expanded model has more parameters (21 versus 5), so the comparison also changes capacity. Dinucleotide frequencies summarize local order and cannot represent all longer-range sequence relationships.

D. *Evidence and conclusion.* The following scores are percentages. Because each test partition contains 2,500 observations from each class, accuracy and balanced accuracy are equal for every reported run. Each entry therefore reports both metrics. The summaries use the sample standard deviation across the three overlapping splits, not a confidence interval. "Freq.+pairs LR" is the additional investigation.

Model	Test input	Seed 11	Seed 22	Seed 33	Mean ± SD
Freq. LR	Original	79.04	78.82	78.76	78.87 ± 0.15
Freq. LR	Shuffled	79.04	78.82	78.76	78.87 ± 0.15
Freq.+pairs LR	Original	97.24	96.92	97.06	97.07 ± 0.16
Freq.+pairs LR	Shuffled	70.38	70.32	70.26	70.32 ± 0.06
MLP	Original	82.14	82.16	83.18	82.49 ± 0.59
MLP	Shuffled	81.64	80.90	82.34	81.63 ± 0.72
CNN	Original	99.36	99.18	99.18	99.24 ± 0.10
CNN	Shuffled	64.58	62.44	62.28	63.10 ± 1.28

Paired accuracy differences below are in percentage points. The shuffle changes subtract original accuracy from shuffled accuracy.

Difference	Seed 11	Seed 22	Seed 33	Mean ± SD
CNN minus MLP (original)	17.22	17.02	16.00	16.75 ± 0.65
Pairs minus frequencies (original)	18.20	18.10	18.30	18.20 ± 0.10
Freq. LR shuffle change	0.00	0.00	0.00	0.00 ± 0.00
Freq.+pairs LR shuffle change	-26.86	-26.60	-26.80	-26.75 ± 0.14
MLP shuffle change	-0.50	-1.26	-0.84	-0.87 ± 0.38
CNN shuffle change	-34.78	-36.74	-36.90	-36.14 ± 1.18

Model	Seed	Best epoch	Epochs run	Val. BCE	Seconds
MLP	11	6	16	0.38343	4.66
CNN	11	31	41	0.02212	77.69
MLP	22	5	15	0.38651	4.47
CNN	22	38	48	0.02210	90.84
MLP	33	5	15	0.38827	4.53
CNN	33	34	44	0.02831	82.96

The experiment took 271.5 seconds on an Intel Core i7-8650U CPU (8 logical CPUs), using two PyTorch computation threads and no GPU. This total includes data preparation, fitting, evaluation, and plotting, and excludes imports and package installation. The per-model times above cover neural training and validation. Freq. LR fitting took 0.013, 0.008, 0.010 seconds for seeds 11, 22, and 33, respectively. Freq.+pairs LR fitting took 0.033, 0.036, 0.040 seconds for seeds 11, 22, and 33, respectively. Software versions were Python 3.12.3, NumPy 1.26.4, pandas 2.1.4, scikit-learn 1.4.1.post1, PyTorch 2.14.1+cpu, and Matplotlib 3.6.3. The runnable notebook is `hw3_question4-tbelcher.ipynb`; dependencies, a standalone script, saved splits, checkpoints, metrics, and histories are provided in the `question4` directory.

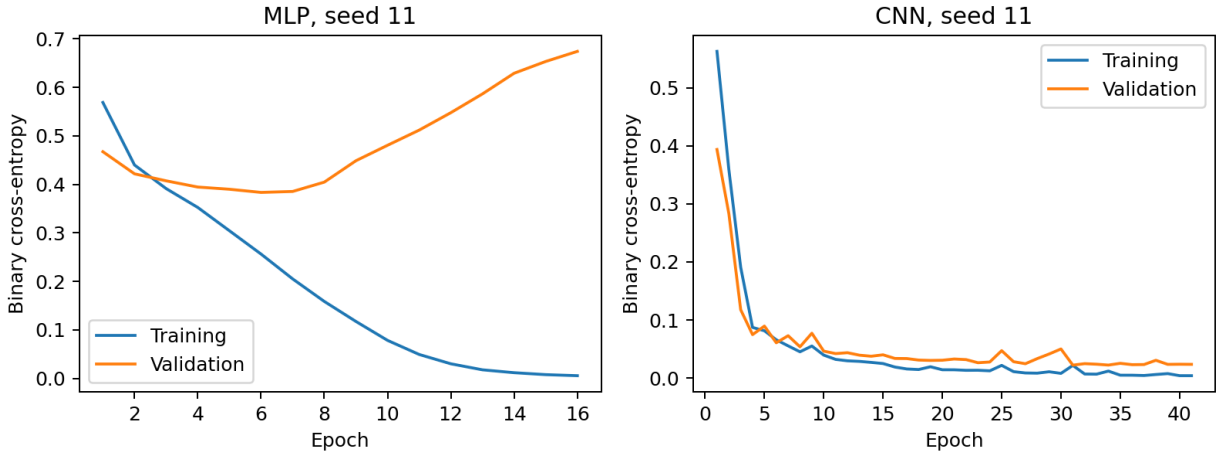


Figure 1: Training and validation loss for the core neural models on seed 11. The minimum validation loss determines the checkpoint.

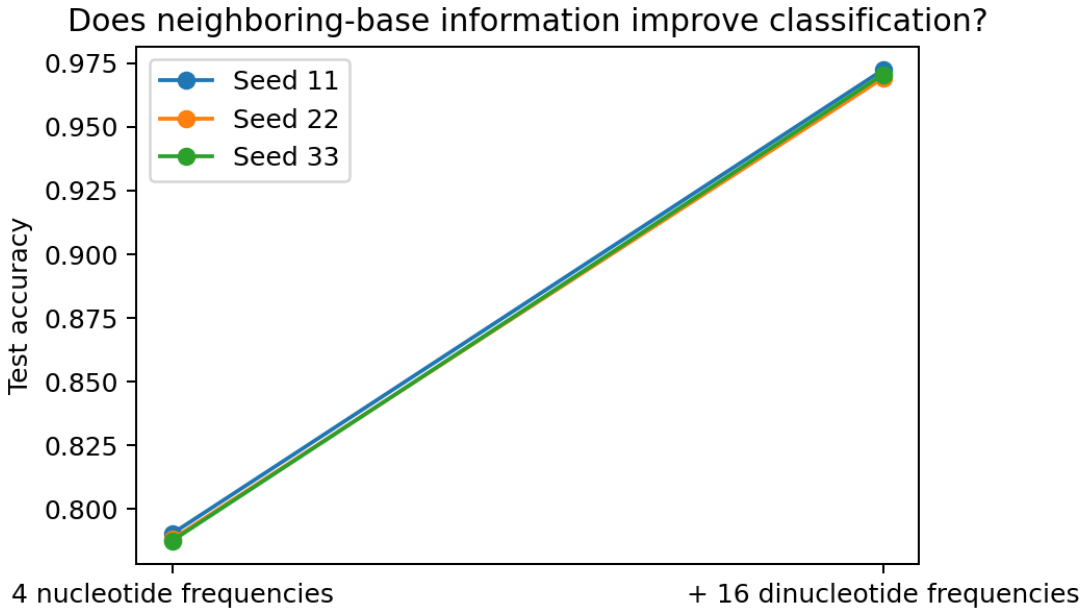


Figure 2: Paired comparison of frequencies alone and frequencies plus adjacent-pair features on the original test sequences.

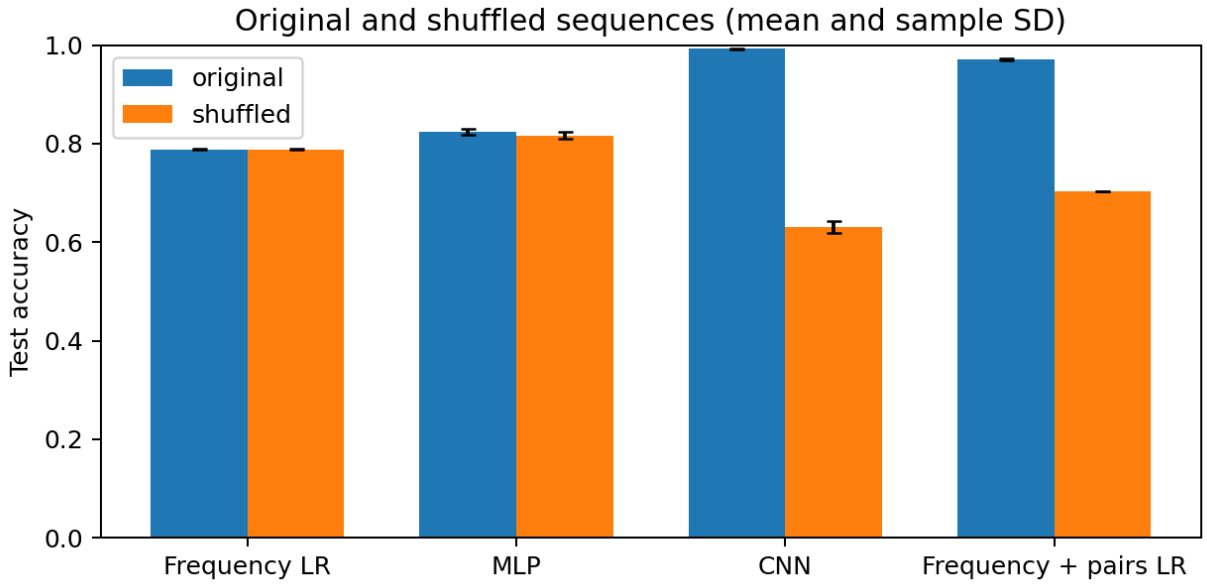


Figure 3: Frozen-model accuracy before and after shuffling. Error bars show sample standard deviations across the three splits.

The results support useful sequence-order information, but challenge my prediction that composition alone would be near chance. Frequency-only logistic regression achieved 78.87% mean accuracy, so composition already distinguishes the classes

substantially. Adding dinucleotide frequencies increased mean accuracy to 97.07%, a paired improvement of 18.20 percentage points. The CNN achieved 99.24%, compared with the MLP's 82.49%, with a mean paired difference of 16.75 percentage points.

Shuffling reduced CNN accuracy by 36.14 percentage points to 63.10%, while the MLP lost only 0.87 percentage points. Frequency-only predictions were exactly unchanged, as expected. Thus the CNN strongly depends on patterns disrupted by shuffling, whereas this MLP is much less sensitive to that perturbation. The MLP's small change does not show that order is uninformative; it may have learned mainly composition or other shuffle-insensitive features under this architecture and training configuration. The seed-11 loss curves also show increasing MLP validation loss after its selected checkpoint, consistent with overfitting.

My strongest supported conclusion is that adjacent-pair statistics add substantial predictive information beyond nucleotide frequencies for these row splits, and the CNN uses information sensitive to position shuffling. These results do not establish that a model interprets biological instructions or that order is universally more important than composition. A plausible alternative is that class-specific collection or sequence-generation artifacts produce the distinguishing statistics. The frozen CNN's deterioration may also partly reflect distribution shift. A follow-up experiment would retrain the same architecture on shuffled training sequences, select checkpoints on shuffled validation sequences, and evaluate on shuffled test sequences. This would help distinguish information lost through shuffling from difficulty processing an unfamiliar input distribution; it is proposed here and was not run.

Random row splits assess performance on held-out rows from this dataset, rather than on new genomic regions or sequence families. Related or overlapping sequences could occur across partitions even without exact duplicates. A stronger evaluation would keep genomic regions or homologous sequence families in separate partitions. It would require source provenance, reference genome and assembly, chromosome and genomic coordinates, and identifiers or similarity-based groupings for sequence families. The supplied sequence and label files do not provide those grouping annotations.

One consequential suggestion I accepted was to compare nucleotide frequencies with frequencies plus neighboring-base information. I checked the feature implementation using ACGTA: its nucleotide frequencies are $(0.4, 0.2, 0.2, 0.2)$, and AC, CG, GT, and TA each have pair frequency $1/4$, with all other pairs zero. Additional checks confirmed one-hot reconstruction, disjoint balanced splits, count preservation after shuffling, and identical frequency-baseline predictions on original and shuffled inputs.